

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра МО ЭВМ**

**ОТЧЕТ**  
**по лабораторной работе №5**  
**по дисциплине «Построение и анализ алгоритмов»**  
**Тема: Ахо-Корасик**  
**Вариант: 7**

Студентка гр. 4343

Попова Е.П.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

## **Задание.**

### **6.1 Ахо-Корасик**

Разработайте программу, решающую задачу точного поиска набора образцов.

Вход:

Первая строка содержит текст ( $T, 1 \leq |T| \leq 100000$ ).

Вторая - число  $n$  ( $1 \leq n \leq 3000$ ), каждая следующая из  $n$  строк содержит шаблон из набора  $P = \{p_1, \dots, p_n\}$   $1 \leq |p_i| \leq 75$ . Все строки содержат символы из алфавита  $\{A, C, G, T, N\}$

Выход:

Все вхождения образцов из  $P$  в  $T$ .

Каждое вхождение образца в текст представить в виде двух чисел -  $i$   $p$

Где  $i$  - позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером  $p$  (нумерация образцов начинается с 1).

Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

### **6.2**

Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с джокером.

В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблоны образцу  $P$  необходимо найти все вхождения  $P$  в текст  $T$ .

Например, образец  $ab??c?$  с джокером  $?$  встречается дважды в тексте  $xabvccbababсах$ .

Символ джокер не входит в алфавит, символы которого используются в  $T$ . Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида  $???$  недопустимы. Все строки содержат символы из алфавита  $\{A, C, G, T, N\}$ .

Вход:

Текст ( $T, 1 \leq |T| \leq 100000$  )

Шаблон ( $P, 1 \leq |P| \leq 40$ )

Символ джокера

Выход:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер).

Номера должны выводиться в порядке возрастания.

### **Выполнение работы.**

Основные структуры данных

6.1 :

- text (str) – текст, в котором выполняется поиск образцов.
- n (int) – количество образцов для поиска.
- patterns (list[str]) – список образцов для поиска.
- Node (class) – вершина бора (автомата).
  - children (dict) – словарь переходов по символам.
  - suffix\_link (Node) – суффиксная ссылка на наибольший собственный суффикс, являющийся префиксом какого-либо образца.
  - output\_link (Node) – терминальная (конечная) ссылка на ближайшую вершину по суффиксным ссылкам, в которой заканчивается хотя бы один образец.
  - patterns (list[int]) – номера образцов, заканчивающихся в данной вершине.
  - depth (int) – глубина вершины в боре (длина префикса).
  - id (int) – уникальный идентификатор вершины.
- root (Node) – корневая вершина бора (начальное состояние автомата).
- node\_count (int) – общее количество вершин в автомате.
- results (list[tuple[int, int]]) – список найденных вхождений в формате (позиция, номер\_образца).

- queue (deque) – очередь для обхода автомата в ширину (BFS) при построении суффиксных ссылок.

6.2:

- text (str) – текст для поиска.
- pattern (str) – шаблон с джокерами.
- wildcard (str) – символ джокера (один символ).
- n (int) – длина текста.
- m (int) – длина шаблона.
- results (list[int]) – список позиций найденных вхождений.

Вариант 7 (Графический интерфейс на CustomTkinter):

1. KmpApp (class) – главный класс приложения, наследующий ctk.CTk.
2. tabview (ctk.CTkTabview) – контейнер с вкладками для разделения задач.
3. entry\_text, entry\_patterns (ctk.CTkEntry) – поля ввода текста и образцов для Ахо-Корасик.
4. entry\_wildcard\_text, entry\_wildcard\_pattern, entry\_wildcard\_char (ctk.CTkEntry) – поля ввода для задачи с джокером.
5. text\_intermediate, text\_wildcard\_intermediate (ctk.CTkTextbox) – текстовые области для вывода промежуточных данных.
6. canvas (tk.Canvas) – холст для графического представления автомата.
7. automaton (AhoCorasick) – экземпляр построенного автомата.

Основные функции.

6.1

- Node.\_\_init\_\_() – конструктор вершины бора, инициализирует поля вершины.
- AhoCorasick.\_\_init\_\_() – конструктор автомата, создаёт корневую вершину.

- `add_pattern(pattern, pattern_num)` – добавляет образец в бор, создавая необходимые вершины и переходы.
- `build_automaton()` – строит суффиксные и терминальные ссылки автомата методом обхода в ширину (BFS).
- `search_with_steps(text)` – выполняет поиск всех вхождений образцов в текст с пошаговой записью процесса.
- `get_automaton_description()` – возвращает текстовое описание автомата (вершины, ссылки, паттерны).
- `get_transitions_description()` – возвращает описание переходов бора.

## 6.2

- `search_with_wildcard()` – выполняет поиск образца с джокером в тексте с подробным выводом промежуточных данных (пошаговое сравнение символов).

Вариант 7 (Графический интерфейс на CustomTkinter):

- `KmpApp.__init__()` – конструктор главного класса приложения, инициализирует окно и вкладки.
- `setup_aho_corasick_tab()` – настройка вкладки Ахо-Корасик.
- `setup_wildcard_tab()` – настройка вкладки поиска с джокером.
- `setup_visualization_tab()` – настройка вкладки визуализации автомата.
- `build_automaton_gui()` – обработчик построения автомата с выводом промежуточных данных.
- `search_patterns()` – обработчик поиска образцов с выводом пошагового процесса.
- `search_with_wildcard()` – обработчик поиска с джокером.
- `draw_automaton()` – отрисовка графического представления автомата на canvas.

## Описание алгоритма

Работа программы начинается с инициализации: считывается текст  $T$  и количество образцов  $n$ , затем считываются сами образцы  $p_1, \dots, p_n$ .

### Фаза 1: Построение бора (trie)

Бор строится путём добавления каждого образца:

1. Начинаем с корневой вершины (root).
2. Для каждого символа образца:
  - a. Если переход по символу не существует, создаём новую вершину.
  - b. Переходим в дочернюю вершину.
3. В конечной вершине добавляем номер образца в список patterns.

### Фаза 2: Построение суффиксных и терминальных ссылок

Используется обход в ширину (BFS):

1. Инициализация: для всех детей корня суффиксная ссылка указывает на корень, терминальная ссылка = None.
2. Для каждой вершины в очереди BFS:
  - a. Для каждого ребёнка вершины:
  - b. Находим суффиксную ссылку: поднимаемся по суффиксным ссылкам родителя, пока не найдём вершину с переходом по символу ребёнка или не достигнем корня.
  - c. Если переход найден: суффиксная ссылка ребёнка = дочерняя вершина по этому переходу.
  - d. Иначе: суффиксная ссылка ребёнка = корень.
  - e. Терминальная ссылка: если суффиксная ссылка содержит паттерны, то терминальная ссылка = суффиксная ссылка, иначе = терминальная ссылка суффиксной ссылки.

### Фаза 3: Поиск вхождений

Автомат обрабатывает текст посимвольно:

1. Начинаем с корневой вершины.
2. Для каждого символа текста:
  - a. Пока текущая вершина  $\neq$  корень и нет перехода по символу: переходим по суффиксной ссылке.
  - b. Если есть переход по символу: переходим в дочернюю вершину.
  - c. Иначе: остаёмся в корне.
  - d. Собираем все паттерны: поднимаемся по терминальным ссылкам, собирая все найденные образцы.
3. Сортируем результаты по позиции, затем по номеру образца.

### 6.2

Алгоритм использует прямой перебор позиций:

1. Для каждой позиции  $i$  в тексте (от 0 до  $n-m$ ):
  - a. Сравниваем шаблон с подстрокой текста, начиная с позиции  $i$ .
  - b. Если символ шаблона = джокер: считаем совпадением.
  - c. Если символ шаблона  $\neq$  джокер и  $\neq$  символу текста: нет совпадения, переходим к следующей позиции.
  - d. Если все символы совпали: добавляем позицию в результаты.
2. Выводим все найденные позиции.

### Вариант 7 (Графическое представление автомата)

Для визуализации автомата строится ориентированный граф:

1. Вершины отображаются как круги с номерами.
2. Переходы бора рисуются сплошными стрелками с метками символов.
3. Суффиксные ссылки рисуются пунктирными линиями.

4. Вершины с паттернами выделяются цветом и рамкой.

Оптимизация алгоритма

Для задачи 6.1 (Ахо-Корасик):

1. Построение бора за  $O(\sum |p_i|)$ . Каждый символ каждого образца обрабатывается один раз при добавлении в бор.
2. Построение суффиксных ссылок за  $O(\sum |p_i|)$ . Обход в ширину гарантирует, что каждая вершина обрабатывается один раз. Амортизированная сложность подъёма по суффиксным ссылкам ограничена глубиной бора.
3. Поиск за  $O(|T| + \text{количество\_вхождений})$ . Каждый символ текста обрабатывается один раз. Подъём по терминальным ссылкам выполняется только при нахождении образцов.
4. Использование словаря для переходов. Словарь Python обеспечивает  $O(1)$  в среднем для проверки наличия перехода и получения дочерней вершины.
5. Предварительное хранение образцов. Список `patterns` позволяет быстро получать длину образца для вычисления позиции вхождения.

Для задачи с джокером:

1. Раннее прерывание сравнения. При первом несовпадении символов (не джокер) цикл прерывается, что ускоряет обработку несовпадающих позиций.
2. Простая реализация. Отсутствие сложных структур данных снижает накладные расходы на инициализацию и хранение.

Для Варианта 7 (Графический интерфейс):

1. Разделение интерфейса на вкладки. Использование `STkTabview` позволяет работать с каждой задачей независимо.

2. Вывод промежуточных данных. Подробное описание построения бора, автомата и процесса поиска удовлетворяет требованиям лабораторной работы.
3. Цветовая индикация результатов. Зелёный цвет при успехе, красный при отсутствии результатов.
4. Модульная архитектура. Алгоритмические функции отделены от функций графического интерфейса.

## Оценка сложности

### 6.1

#### Теоретическая сложность:

- Временная сложность:  $O(|T| + \sum |p_i| + \text{количество\_вхождений})$ .
  - Построение бора:  $O(\sum |p|)$ , где  $\sum |p|$  – суммарная длина всех образцов.
  - Построение суффиксных ссылок:  $O(\sum |p|)$ .
  - Поиск в тексте:  $O(|T| + \text{количество\_вхождений})$ .
  - Сортировка результатов:  $O(K \log K)$ , где  $K$  – количество вхождений.
- Пространственная сложность:  $O(\sum |p| \times |\text{алфавит}|)$ .
  - Бор содержит не более  $\sum |p_i| + 1$  вершин.
  - Каждая вершина хранит словарь переходов (в худшем случае  $|\text{алфавит}|$  элементов).
  - Суффиксные и терминальные ссылки:  $O(\sum |p|)$ .

#### Практическая сложность:

Алгоритм Ахо-Корасик эффективно обрабатывает тексты длиной до 100 000 символов и до 3 000 образцов длиной до 75 символов. Суммарная длина образцов не превышает 225 000 символов, что обеспечивает построение автомата за доли секунды.

## Задача с джокером

#### Теоретическая сложность:

- Временная сложность:  $O(|T| \times |P|)$ , где  $|T|$  – длина текста,  $|P|$  – длина шаблона.
  - Внешний цикл:  $O(|T| - |P| + 1)$  итераций.
  - Внутренний цикл:  $O(|P|)$  сравнений в худшем случае.
- Пространственная сложность:  $O(1)$  дополнительных структур (кроме хранения входных данных).

Практическая сложность:

При  $|T| \leq 100\,000$  и  $|P| \leq 40$  алгоритм выполняет до 4 000 000 операций, что выполняется за доли секунды.

Вариант 7 (Графическое представление автомата):

- Временная сложность построения графа:  $O(V + E)$ , где  $V$  – количество вершин,  $E$  – количество рёбер.
- Пространственная сложность хранения графа:  $O(V + E)$ .
- Отрисовка графа:  $O(V + E)$  с использованием библиотеки `tkinter.Canvas`.

Оценка по памяти

6.1

- Бор:  $O(\sum |p_i| \times |\text{алфавит}|)$  – при  $\sum |p| = 225\,000$  и  $|\text{алфавит}| = 5$  занимает порядка нескольких мегабайт.
- Суффиксные и терминальные ссылки:  $O(\sum |p_i|)$ .
- Список образцов:  $O(\sum |p|)$ .
- Результаты поиска:  $O(K)$ , где  $K$  – количество вхождений.

Задача с джокером:

- Текст и шаблон:  $O(|T| + |P|)$ .
- Результаты:  $O(|T|)$  в худшем случае.

Вариант 7 (Графическое представление):

- Структуры данных графа:  $O(V + E)$ .

- Буфер отрисовки: зависит от библиотеки, для автоматов с несколькими тысячами вершин занимает десятки мегабайт.

## Тестирование

Экспериментальные результаты подтверждают корректность работы алгоритма на различных тестовых случаях, включая:

- Тексты с несколькими вхождениями разных образцов
- Перекрывающиеся вхождения
- Образцы, не найденные в тексте
- Тексты с джокерами
- Граничные случаи (пустые строки, один символ)

Таблица 1: Тестирование программы 6.1

| Входные данные         | Выходные данные                                  |
|------------------------|--------------------------------------------------|
| NTAG<br>TAGT,TAG,T     | 2 2, 2 3                                         |
| ababab<br>ab,ba,a      | 1 1, 1 3, 2 2, 3 1, 3 3, 4 2, 5 1, 5 3, 6 2      |
| acdef<br>abc, def, xyz | 1 1, 4 2                                         |
| aaaa<br>a, aa, aaa     | 1 1, 1 2, 1 3, 2 1, 2 2, 3 1, 3 3, 4 1, 4 2, 5 1 |
| abc<br>abc             | 1 1                                              |

Таблица 2: Тестирование программы 6.2

| Входные данные        | Выходные данные |
|-----------------------|-----------------|
| ACTANCA<br>A\$A<br>\$ | 1               |
| xabuccsbababcaх       | 2, 9            |

|                    |       |
|--------------------|-------|
| ab??c?<br>?        | 0     |
| aaaa<br>a\$a<br>\$ | 1,2,3 |

Таблица 3: Тестирование промежуточных данных

| Входные данные  | Выходные данные |
|-----------------|-----------------|
| TAGT,TAG,T      | 5               |
| he,she,his,hers | 10              |
| a,b,c           | 4               |
| tag,t,ag        | 4               |

### Вывод

В ходе выполнения лабораторной работы был успешно реализован алгоритм Ахо-Корасик для точного поиска множества образцов в тексте, включающий построение бора, вычисление суффиксных и терминальных ссылок, а также поиск всех вхождений с сортировкой результатов. Дополнительно была решена задача поиска образца с джокером методом прямого перебора позиций с пошаговым сравнением символов. Для индивидуального варианта 7 был разработан графический интерфейс на базе библиотеки CustomTkinter, обеспечивающий интерактивный ввод данных, вывод промежуточных данных (построение бора, описание автомата, пошаговый процесс поиска) и наглядное графическое представление конечного автомата с отображением вершин, переходов и суффиксных ссылок. В процессе разработки были использованы ключевые оптимизации алгоритма: построение бора за  $O(\sum |p_i|)$ , построение суффиксных ссылок за  $O(\sum |p_i|)$  методом обхода в ширину, поиск в тексте за  $O(|T| + \text{количество\_вхождений})$  с использованием терминальных ссылок для эффективного сбора всех найденных образцов. Теоретическая сложность реализованных алгоритмов составляет  $O(|T| + \sum |p_i| +$

количество\_вхождений) по времени и  $O(\sum |p_i| \times |\text{алфавит}|)$  по памяти для задачи Ахо-Корасик,  $O(|T| \times |P|)$  по времени и  $O(|T| + |P|)$  по памяти для задачи с джокером, что подтверждает высокую практическую эффективность методов для задач строкового поиска и делает их применимыми для обработки больших объёмов текста.